DATABASE MANAGEMENT SYSTEM

Database Design Analysis

Redundancy, anomalies, functional dependencies and design improvements

Submitted By	
Name	Palle Sai Rithesh
Roll Number	2511021240013
Course	BCA Game Development
Subject	Database Management System
Assignment	Assignment 3 — Database Design Analysis
Project	Online Shopping System

Assignment 3 — requirements

- Examine your database design to identify data redundancy and potential insertion, deletion, and update anomalies.
- Identify the functional dependencies between attributes.
- Explain how poor database design can lead to these anomalies.
- Propose suitable improvements to your database design based on your analysis.

01 TASK 1

Redundancy and Anomalies: What Can Go Wrong

A single flat sheet makes the problems easy to see before the real design is examined.

Data redundancy means the same fact is stored in more than one place. Redundancy wastes space, but the bigger harm is that copies can disagree. Three kinds of **anomaly** follow from it: an **update anomaly** (a change must be repeated in many rows), an **insertion anomaly** (a fact cannot be stored without an unrelated one) and a **deletion anomaly** (removing one fact accidentally destroys another).

Imagine the whole shop kept in one table, *Order_Master*, with one row per product in an order. The key is (order_id, product_name). The customer's address is left out of the figure only to save width. Yellow cells are repeated facts.

order_id	customer_name	customer_email	product_name	category	price	qty
1	Aarav Sharma	aarav.sharma@example.com	Wireless Bluetooth Earbuds	Electronics	1499.00	1
1	Aarav Sharma	aarav.sharma@example.com	SQL Fundamentals for Beginners	Books	599.00	1
6	Aarav Sharma	aarav.sharma@example.com	20000 mAh Power Bank	Electronics	1899.00	1
10	Priya Nair	priya.nair@example.com	Wireless Bluetooth Earbuds	Electronics	1499.00	2
10	Priya Nair	priya.nair@example.com	Smart Fitness Band	Electronics	2499.00	1
12	Meera Kulkarni	meera.kulkarni@example.com	Introduction to Game Design	Books	850.00	1

Table 1 — The all-in-one Order_Master sheet (sample rows from the shop's data).

Problem	What happens in Order_Master
Redundancy	Aarav's name and e-mail are stored three times, 'Electronics' four times, and the earbuds' price twice.
Update anomaly	Aarav changes his e-mail: three rows must be edited. If one is missed, the shop holds two e-mail addresses for one person. If the earbuds drop to ₹1,399, both rows must change or the same product has two prices.
Insertion anomaly	A new product (Gaming Mouse) cannot be stored until someone orders it, because every row needs an order_id, which is part of the key and cannot be empty. Likewise customer Lakshmi Venkatesh, who has not ordered yet, cannot be recorded.
Deletion anomaly	Meera cancels order 12 and its row is deleted. The only record of *Introduction to Game Design* and of Meera herself disappears with it.

> **Note: Root cause.** The sheet mixes facts about four different things (customer, order, product, category) in one table. Each fact then depends on only part of the key, or on another non-key column. Section 3 names these dependencies precisely.

02 TASK 2

Examining the Assignment 2 Design

The eight-table relational schema is checked table by table for redundancy and anomaly risks.

The design examined is the relational schema of Assignment 2: **customers, categories, products, orders, order_items, payments, shipments** and **reviews**. It already avoids the worst problem above, because customers, products, categories and orders each have their own table. Some decisions are also correct on purpose:

- **order_items.unit_price** repeats a price that also exists in products, but it is a different fact: the price *at the time of purchase*. Without it, a later price change would silently rewrite every past order. This is not redundancy.
- **Several payments per order** allows retries and refunds to be recorded without overwriting history.
- **order_items** correctly resolves the many-to-many link between orders and products.

The examination still finds the following weaknesses.

ID	Where	Weakness	Kind	Risk
F1	customers.address	One VARCHAR holds house, street, city, state and pincode, and a customer can have only one. A second address means a duplicate customer row or overwriting the first. City and state text repeats for every customer in the same city and may be spelt differently (Bengaluru, Bangalore).	Redundancy, update and insertion anomaly	High
F2	orders	An order does not record where it was delivered. If a customer moves, every old order appears to have gone to the new address.	Update anomaly, lost history	High
F3	orders.total_amount	The total is stored but can also be calculated from order_items. Changing a line without changing the total leaves the two out of step (demonstrated below).	Derived-data redundancy, update anomaly	Medium
F4	orders.status and shipments.delivery_status	Both columns record delivery progress (Shipped, Delivered). They can disagree: order Delivered, shipment still Out for delivery.	Redundancy, update anomaly	Medium
F5	status, method and rating columns	Free-text columns with no allowed list accept ‘delivered’, ‘Delivered ’ or ‘DELIVERED’, and a rating of 9.	Inconsistent data	Medium
F6	order_items	Only the surrogate key is unique, so the same product can appear twice in one order. The real key, (order_id, product_id), is not enforced.	Duplicate rows	Medium
F7	reviews	The same customer can review the same product many times; (customer_id, product_id) is not enforced.	Duplicate rows	Low
F8	shipments.carrier	Carrier names are free text repeated on every shipment. Spelling differences split reports. No other carrier attribute exists, so this is a minor risk and is accepted.	Minor redundancy	Low

Table 2 — Findings from the examination of the Assignment 2 design.

Demonstration of F3: an update anomaly

A customer changes the quantity of the SQL book in order 1 from 1 to 3. Only the line is updated; the stored total is forgotten.

```
UPDATE order_items SET quantity = 3 WHERE order_id = 1 AND product_id = 8;
SELECT o.order_id, o.total_amount AS stored_total, SUM(i.quantity * i.unit_price) AS calculated_total
FROM orders o JOIN order_items i ON i.order_id = o.order_id
WHERE o.order_id = 1
GROUP BY o.order_id, o.total_amount;
```

OUTPUT

order_id	stored_total	calculated_total
1	2098	3296

(1 row)

Explanation: The order claims ₹2,098.00 while its own lines add up to ₹3,296.00. Any revenue report that reads total_amount is now wrong by ₹1,198.00, and no error was raised. (The test update was rolled back.)

03 TASK 3

Functional Dependencies

A functional dependency X → Y means that each value of X determines exactly one value of Y.

3.1 Dependencies in the Order_Master sheet

For the flat relation R(order_id, order_date, customer_id, customer_name, customer_email, customer_address, product_id, product_name, category_id, category_name, price, quantity), the key is (order_id, product_id) and the dependencies are:

Dependency	Type	Why it is a problem
order_id → order_date, customer_id	Partial (part of the key)	Order facts repeat for every product in the order
customer_id → customer_name, customer_email, customer_address	Transitive (via order_id → customer_id)	Customer facts repeat for every order and every line
product_id → product_name, category_id, price	Partial (part of the key)	Product facts repeat for every order that contains the product
category_id → category_name	Transitive (via product_id → category_id)	Category name repeats for every product
(order_id, product_id) → quantity	Full dependency on the key	Correct: quantity belongs to this order line

Removing the partial dependencies gives second normal form (2NF) and removing the transitive ones gives third normal form (3NF):

Step	Action	Relations obtained
1NF	Values are atomic and every row is unique	R as above (key: order_id, product_id)
2NF	Move attributes that depend on only part of the key	Orders(order_id, order_date, customer_id, customer_*) · Products(product_id, product_name, category_id, category_name, price) · Order_Items(order_id, product_id, quantity)
3NF	Move attributes that depend on a non-key attribute	Customers(customer_id, name, email, address) · Categories(category_id, category_name) · plus the tables above

This decomposition produces the core of the Assignment 2 schema, which is why the design avoids the anomalies of Table 1.

3.2 Dependencies in the Assignment 2 tables

Table	Functional dependencies	Candidate keys	Normal form
customers	customer_id → name, email, phone, address email → customer_id (e-mail is unique) Inside address: pincode → city, state	customer_id, email	3NF, but address is not atomic (1NF in spirit)
categories	category_id → category_name, description	category_id	BCNF
products	product_id → category_id, name, price, stock_qty	product_id	BCNF
orders	order_id → customer_id, order_date, status, total_amount Derived rule: total_amount = Σ quantity × unit_price over the order's items	order_id	BCNF; derived value (F3)
order_items	order_item_id → order_id, product_id, quantity, unit_price (order_id, product_id) → quantity, unit_price (intended, not enforced)	order_item_id; (order_id, product_id)	BCNF once the key is enforced
payments	payment_id → order_id, payment_method, amount, payment_status	payment_id	BCNF
shipments	shipment_id → order_id, tracking_no, carrier, delivery_status tracking_no → shipment_id	shipment_id, tracking_no	BCNF
reviews	review_id → customer_id, product_id, rating, review_text (customer_id, product_id) → rating, review_text (intended)	review_id; (customer_id, product_id)	BCNF once the key is enforced

Table 3 — Functional dependencies of the Assignment 2 schema.

> **Note: Reading the table.** A table is in BCNF when every determinant (left side of a dependency) is a candidate key. Most tables already satisfy this. The weaknesses are not extra determinants but a non-atomic address column (F1), an overlap between tables (F3, F4) and unenforced keys (F6, F7).

04 TASK 4

How Poor Design Leads to Anomalies

Every anomaly can be traced to a fact that is stored twice or bundled with an unrelated fact.

Design mistake	Anomaly it causes	Mechanism
The same fact stored in several rows or tables	Update anomaly	A change must be applied to every copy. Missing one leaves contradictory data, and nothing in the database notices.
Facts about different subjects bundled in one table	Insertion and deletion anomaly	A fact can exist only while its row partner exists, so adding one fact needs an unrelated fact and removing one deletes another.
A non-key column that depends on another non-key column	Update anomaly (transitive dependency)	The determinant column is copied into many rows, together with everything it determines.
A column that depends on only part of a composite key	Redundancy (partial dependency)	The part of the key repeats, so the dependent value repeats with it.
Derived values stored next to their source	Inconsistency	Two columns say the same thing in different ways and drift apart.
No UNIQUE, CHECK or domain rules	Duplicate and invalid rows	The database cannot refuse a bad value, so every program must remember to.

Scenarios in the Assignment 2 design

Scenario	What happens	Business consequence
Priya moves house and updates her profile (F1, F2)	customers.address is overwritten. Her earlier orders still point to her customer row, so they now seem to have been delivered to the new address.	Disputes about past deliveries cannot be settled; returns and invoices show the wrong address.
A customer wants a home and an office address (F1)	Only one address column exists, so the shop either overwrites it or creates a second customer row with the same name, e-mail and phone.	Duplicate customers, split order history, and two logins for one person.
A quantity is edited in an order line (F3)	order_items changes but orders.total_amount does not.	Revenue reports are wrong and the payment amount no longer matches the order.
A courier marks a parcel Delivered (F4)	shipments.delivery_status is updated but orders.status still says Shipped.	The customer sees two different statuses; the support team cannot tell which is right.
The same product is added twice to one order (F6)	Two rows exist for (order_id, product_id).	Stock is reduced twice and quantity-based reports double count.
A customer posts three reviews of one product (F7)	Three rows are accepted.	The average rating is skewed by one person.

> **Note: Why it matters at scale.** With a handful of rows these errors are easy to fix by hand. With millions of orders, each unnoticed inconsistency spreads into reports, invoices and customer messages, and finding the correct value later is often impossible.

05 TASK 5

Proposed Improvements to the Design

Each finding is answered by a structural change or a constraint.

Finding	Improvement	Result
F1 non-atomic, single address	Create **addresses** (one row per saved address, with customer_id) and remove customers.address. Split city and state into **pincodes** because pincode → city, state.	Several addresses per customer; city and state stored once; 3NF
F2 delivery address not on the order	Add **orders.address_id** referencing addresses.	Each order keeps its own delivery address, even after the customer moves
F3 derived total	Keep total_amount as a deliberate snapshot of what was charged (it may later include discount or shipping) but reconcile it with the order lines using a check query (Assignment 7).	Redundancy is documented and monitored instead of accidental
F4 two delivery statuses	Restrict orders.status to Placed, Confirmed, Completed, Cancelled. Delivery stages (Packed, Shipped, Out for delivery, Delivered) live only in **shipments**.	Each fact has one home
F5 free-text domains	Add **CHECK** constraints for status, payment method, delivery status and rating range.	Invalid values are rejected
F6, F7 unenforced keys	Add **UNIQUE (order_id, product_id)** and **UNIQUE (customer_id, product_id)**.	No duplicate lines or reviews
F8 carrier text	Accepted. Shipments keep the carrier name; a carriers table is only worth adding if carrier details (contact, tracking link) are needed.	Documented trade-off
General	Add NOT NULL, DEFAULT values and ON DELETE rules (cascade for dependants, restrict for financial and delivery records).	Safer inserts and deletes

Table 4 — Findings and the improvements adopted.

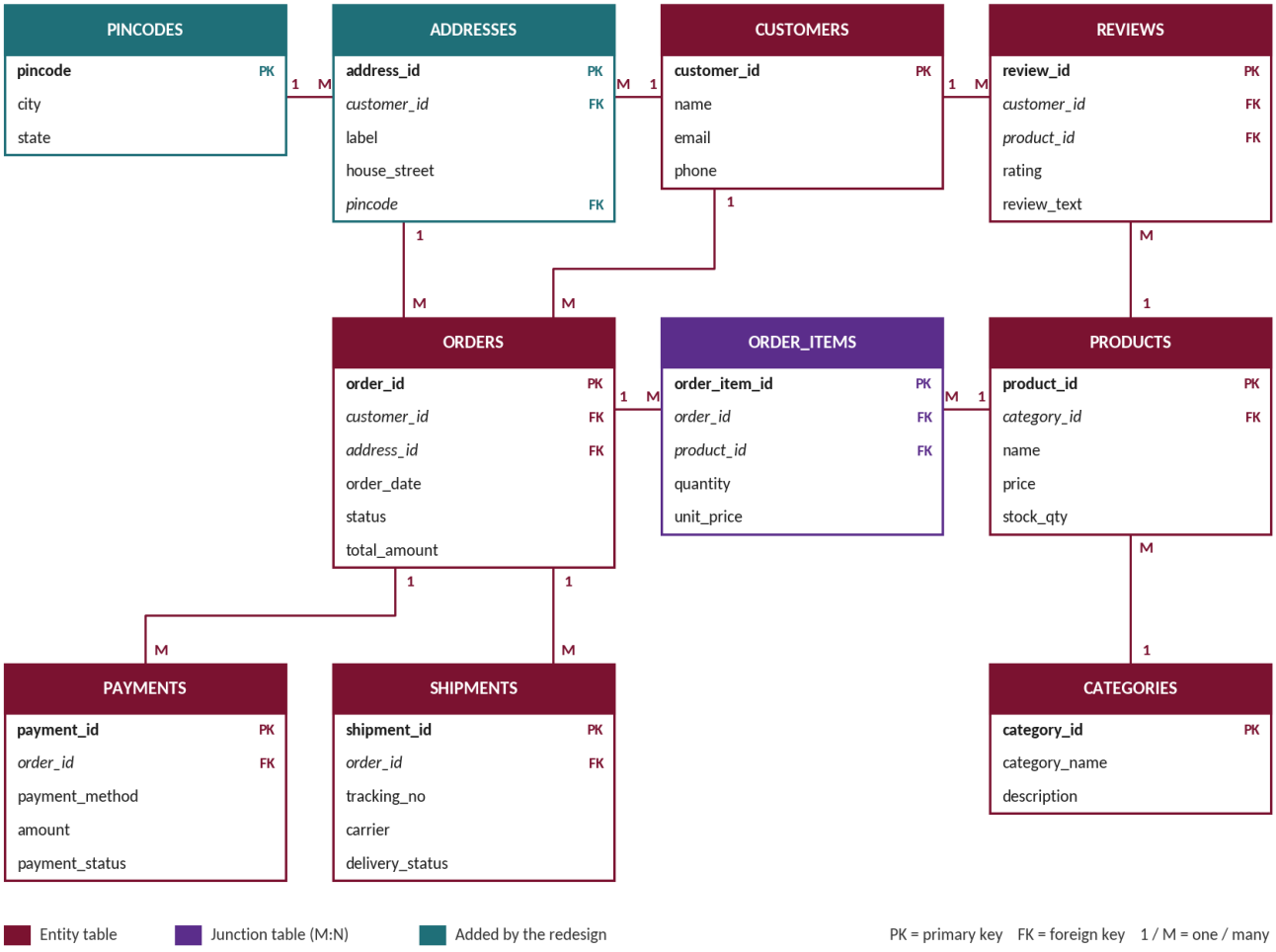


Figure 1 — Improved relational schema. Teal tables are new; the purple table resolves the many-to-many link.

A migration sketch for an existing database:

```
-- 1. New tables
CREATE TABLE pincodes  (pincode CHAR(6) PRIMARY KEY, city VARCHAR(60) NOT NULL, state VARCHAR(60) NOT NULL);
CREATE TABLE addresses (address_id INT AUTO_INCREMENT PRIMARY KEY, customer_id INT NOT NULL,
                        label VARCHAR(20) NOT NULL DEFAULT 'Home', house_street VARCHAR(150) NOT NULL,
                        pincode CHAR(6) NOT NULL,
                        FOREIGN KEY (customer_id) REFERENCES customers (customer_id) ON DELETE CASCADE,
                        FOREIGN KEY (pincode) REFERENCES pincodes (pincode));
-- 2. Move each customer's old address text into addresses (split into street and pincode)
-- 3. Point every existing order at its customer's address, then tighten the column
ALTER TABLE orders ADD COLUMN address_id INT NULL;
UPDATE orders SET address_id = (SELECT MIN(address_id) FROM addresses a WHERE a.customer_id = orders.customer_id);
ALTER TABLE orders MODIFY address_id INT NOT NULL,
      ADD CONSTRAINT fk_orders_address FOREIGN KEY (address_id) REFERENCES addresses (address_id);
-- 4. Remove the old column and add the UNIQUE and CHECK rules (see Assignment 6)
ALTER TABLE customers DROP COLUMN address;
```

Normal form of the improved tables

Table	Key dependency	Normal form
customers	customer_id → name, email, phone	BCNF
pincodes	pincode → city, state	BCNF
addresses	address_id → customer_id, label, house_street, pincode	BCNF (city and state no longer depend on a non-key column)
categories, products, order_items, payments, shipments, reviews	Unchanged from Table 3, with UNIQUE keys now enforced	BCNF
orders	order_id → customer_id, address_id, order_date, status, total_amount	BCNF; total_amount is a documented snapshot

Anomalies before and after

Anomaly	Assignment 2 design	Improved design
Update: customer changes address	Overwrites the only address; old orders lose their destination	Add or edit a row in addresses; orders keep their address_id
Update: city spelt differently	Possible in every address string	City stored once per pincode
Insertion: second address	Needs a duplicate customer row	One new row in addresses
Deletion: customer removed	Customer and address vanish together	Cascade removes only addresses of a customer with no orders; customers with orders cannot be deleted
Update: delivery status	Two columns can disagree	Only shipments holds delivery status
Duplicate order line or review	Accepted	Rejected by UNIQUE
Invalid status, method or rating	Accepted	Rejected by CHECK

Note: Trade-offs. The improved design needs more joins (for example four tables to print a delivery address), and a pincode may in practice cover more than one locality. Both costs are small compared with the protection against inconsistent data. The design is implemented in Assignments 5 to 9.